PROJECT DOCUMENTATION

CampusShield

A campus security monitoring console for managing exam-integrity shields across buildings, labs, and computers. Integration & requirements reference for developers.

STACK
React 18 · TypeScript · Vite

STATUS
Frontend reference app

PERSISTENCE
localStorage (Supabase-ready)

AUDIENCE
Integrating developers

VERSION
1.0

DATE
August 2026

Contents

1. Quick start

Requirements: Node.js 18+ and npm.

```
npm install
npm run dev
```

The dev server starts on the URL printed in the terminal (Vite default: `http://localhost:5173`).

For a production build:

```
npm run build
npm run preview
```

2. Demo credentials

The app ships with a single seeded demo user:

FIELD	VALUE
Email	`admin@campusshield.edu`
Password	`Admin@123`

There is a "Use demo credentials" shortcut on the login page. Authentication is client-side only (see Section 5).

3. Tech stack

CONCERN	CHOICE
Build tool	Vite 5
UI framework	React 18
Language	TypeScript 5
Styling	Tailwind CSS 3 + hand-written CSS variables in `src/index.css`
Routing	react-router-dom 7
Icons	lucide-react
Persistence	`localStorage` (via `src/services/storage.ts`)
Backend client	`@supabase/supabase-js` (installed, not yet used)

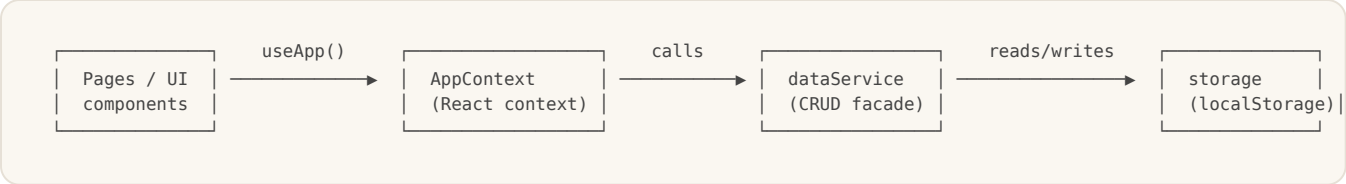
No UI kit, state library, or form library is used — the component library and state layer are hand-rolled and live in this repo.

4. Project structure

```
project/
├── public/
│   ├── logo.jpg
│   └── placement-bg_1.png        # decorative asset used on the login page
├── src/
│   ├── App.tsx                   # Router + route guards (ProtectedRoute / PublicRoute)
│   ├── main.tsx                  # React root
│   ├── index.css                 # Global styles + design-token CSS variables
│   ├── vite-env.d.ts
│   ├── assets/
│   │   └── image.png
│   ├── components/
│   │   ├── layout/
│   │   │   ├── AppShell.tsx      # Sidebar + TopBar + content outlet
│   │   │   ├── Sidebar.tsx
│   │   │   └── TopBar.tsx        # Search, notifications, user menu
│   │   └── ui/                   # Reusable primitives (see Section 9)
│   │       ├── Badge.tsx
│   │       ├── Button.tsx
│   │       ├── ConfirmDialog.tsx
│   │       ├── EmptyState.tsx
│   │       ├── ErrorState.tsx
│   │       ├── GlassCard.tsx
│   │       ├── Modal.tsx
│   │       ├── PageHeader.tsx
│   │       ├── SearchBar.tsx
│   │       ├── ShieldStatus.tsx
│   │       ├── Skeleton.tsx
│   │       ├── StatCard.tsx
│   │       └── Toast.tsx
│   ├── context/
│   │   └── AppContext.tsx        # Single global provider (auth + data + toasts)
│   ├── pages/
│   │   ├── LoginPage.tsx
│   │   ├── DashboardPage.tsx
│   │   ├── ExamShieldPage.tsx
│   │   ├── AlertsPage.tsx
│   │   ├── AuditLogsPage.tsx
│   │   ├── SettingsPage.tsx
│   │   ├── InfrastructurePage.tsx
│   │   ├── BuildingsPage.tsx
│   │   ├── LabsPage.tsx
│   │   └── ComputersPage.tsx
│   ├── services/
│   │   ├── dataService.ts        # CRUD facade over storage
│   │   ├── storage.ts            # localStorage wrapper + key registry
│   │   └── seedData.ts           # Initial demo data
│   └── types/
│       └── index.ts              # All shared TypeScript types
├── index.html
├── vite.config.ts
├── tailwind.config.js
├── postcss.config.js
├── tsconfig.json
├── tsconfig.app.json
├── tsconfig.node.json
├── eslint.config.js
└── package.json
```

5. Architecture overview

The app follows a simple three-layer architecture. There is no backend round-trip today — everything runs in the browser.



1. **`storage.ts`** — a thin typed wrapper around `localStorage` with a `campusshield:` key prefix and a central `STORAGE_KEYS` registry.
2. **`dataService.ts`** — the only module that touches `storage`. Exposes typed CRUD methods for every entity and handles cascading deletes (e.g. deleting a building removes its labs, their computers, and related alerts). Also seeds demo data on first load.
3. **`AppContext.tsx`** — a single React context that loads all entities into state, exposes mutate functions, writes audit-log entries on every mutation, and manages toasts + theme application. Pages consume it via the `useApp()` hook.

Authentication is a single hard-coded demo user validated in `dataService.authenticate`. The session (user + `remember` flag) is stored in `localStorage`. `App.tsx` guards routes with `ProtectedRoute` / `PublicRoute`.

Audit logging is automatic: every mutating action in `AppContext` calls `dataService.addAuditLog(...)` with the action, entity, entity id, and a description, attributed to the current user (or `"System"`).

6. Data model

All types live in `src/types/index.ts`. IDs are strings.

6.1 Entities

ENTITY	TYPE	KEY FIELDS	RELATIONSHIPS
User	`User`	`id`, `name`, `email`, `password`, `role`, `avatarColor`	—
Building	`Building`	`id`, `name`, `code`, `location`, `description`, `status`, `createdAt`	has many Labs
Lab	`Lab`	`id`, `name`, `code`, `buildingId`, `capacity`, `status`, `createdAt`	belongs to a Building; has many Computers
Computer	`Computer`	`id`, `name`, `assetId`, `labId`, `os`, `ipAddress`, `status`, `shieldEnabled`, `createdAt`	belongs to a Lab
Alert	`Alert`	`id`, `buildingId`, `labId?`, `computerId?`, `type`, `severity`, `description`, `attachment?`, `status`, `createdAt`, `updatedAt`	linked to a Building and optionally a Lab/Computer
AuditLog	`AuditLog`	`id`, `timestamp`, `user`, `action`, `entity`, `entityId?`, `description`	—
Settings	`Settings`	`theme`, `notifications{}`, `security{}`	per-installation
Session	`Session`	`userId`, `remember`	—

6.2 Status / severity unions

TYPE	VALUES
`BuildingStatus`	`active` | `maintenance` | `inactive`
`LabStatus`	`active` | `maintenance` | `inactive`
`ComputerStatus`	`active` | `maintenance` | `offline`
`ShieldStatus`	`protected` | `partially` | `unprotected`
`AlertSeverity`	`low` | `medium` | `high` | `critical`
`AlertStatus`	`open` | `investigating` | `resolved` | `dismissed`
`Theme`	`light` | `dark` | `system`

6.3 localStorage keys

All keys are prefixed with `campusshield:` (see `storage.ts`).

CONSTANT (`STORAGE_KEYS`)	KEY	STORED TYPE
`USER`	`user`	`User`
`SESSION`	`session`	`Session`
`BUILDINGS`	`buildings`	`Building[]`
`LABS`	`labs`	`Lab[]`
`COMPUTERS`	`computers`	`Computer[]`
`ALERTS`	`alerts`	`Alert[]`
`AUDIT_LOGS`	`auditLogs`	`AuditLog[]`
`SETTINGS`	`settings`	`Settings`
`SEEDED`	`seeded`	`boolean` (flag)

7. State management (useApp)

`AppContext` is the single source of truth. Consume it anywhere inside `<AppProvider>` with:

```
import { useApp } from '@/context/AppContext';

function MyComponent() {
  const {
    currentUser, isAuthenticated, login, logout,
    buildings, labs, computers, alerts, auditLogs, settings,
    createBuilding, updateBuilding, deleteBuilding,
    createLab, updateLab, deleteLab,
    createComputer, updateComputer, deleteComputer,
    setShield,
    createAlert, updateAlert, setAlertStatus,
    saveSettings,
    refresh, loading,
    toasts, showToast, dismissToast,
  } = useApp();
}
```

Notes for integrators:

- **`refresh()`** re-reads every entity from `dataService` into state. It is called automatically after every mutation, so you rarely call it manually.
- **`loading`** is `true` only during the initial synchronous load on mount.
- **Toasts**: call `showToast(message, 'success' | 'error' | 'info')`. Toasts auto-dismiss after ~3.5s; `dismissToast(id)` removes one early.
- **`setShield(computerIds, enabled)`** is the bulk toggle used by the Exam Shield page; it flips `shieldEnabled` on the given computers and logs an audit entry.
- Every mutation function writes an `AuditLog` entry attributed to `currentUser.name` (or `"System"`).

8. Pages and routes

Defined in `src/App.tsx`. All protected routes render inside `<AppShell>` (sidebar + topbar).

PATH	PAGE	AUTH	PURPOSE
`/login`	`LoginPage`	no	Sign in with demo credentials
`/dashboard`	`DashboardPage`	yes	Overview stats, shield distribution, severity bars, activity
`/exam-shield`	`ExamShieldPage`	yes	Browse building → lab → computer tree; toggle shields (bulk)
`/alerts`	`AlertsPage`	yes	List, filter, create, update, resolve/dismiss security alerts
`/audit-logs`	`AuditLogsPage`	yes	Timeline of every mutating action
`/settings`	`SettingsPage`	yes	Theme, notifications, security toggles
`/infrastructure`	`InfrastructurePage`	yes	Read-only building/lab/computer hierarchy overview

- `/` redirects to `/dashboard`.
- Unknown paths redirect to `/dashboard`.
- `ProtectedRoute` redirects unauthenticated users to `/login` (preserving the intended destination in `location.state.from`).
- `PublicRoute` redirects already-authenticated users away from `/login`.
- `ScrollToTop` resets scroll on every route change.

8.1 Adding a new page

1. Create `src/pages/MyPage.tsx` exporting a `MyPage` component.

2. Add a route in `src/App.tsx` inside the protected `<AppShell>` block:

```
<Route path="/my-page" element={<MyPage />} />
```

3. Add a sidebar entry in `src/components/layout/Sidebar.tsx` and a topbar title mapping in `src/components/layout/TopBar.tsx`.

9. UI component library

All primitives live in `src/components/ui/` and are plain TypeScript + CSS classes from `src/index.css` (no Tailwind utility classes inside components). Import via the `@/` alias.

COMPONENT	FILE	PROPS SUMMARY
`Button`	`Button.tsx`	`variant?: 'primary' | 'secondary' | 'danger' | 'ghost'`, `size?: 'sm' | 'md' | 'lg'`, plus native button props.
`Badge`	`Badge.tsx`	Status/severity pills.
`Modal`	`Modal.tsx`	Sizes `sm` / `md` / `lg`; header/body/footer slots.
`ConfirmDialog`	`ConfirmDialog.tsx`	`open`, `title`, `message`, `confirmLabel`, `onConfirm`, `onCancel`, `danger?`.
`GlassCard`	`GlassCard.tsx`	Frosted-glass surface (`glass` variant) or plain surface.
`PageHeader`	`PageHeader.tsx`	`title`, `description`, actions slot.
`SearchBar`	`SearchBar.tsx`	Controlled search input with clear button.
`ShieldStatus`	`ShieldStatus.tsx`	Renders `protected` / `partially` / `unprotected` with colored dot + label, sizes `sm` / `md` / `lg`.
`StatCard`	`StatCard.tsx`	Dashboard KPI card with icon, value, sublabel; color variants `green` / `amber` / `red`.
`Skeleton`	`Skeleton.tsx`	Shimmer loading placeholders (card / row / block).
`EmptyState`	`EmptyState.tsx`	`icon`, `title`, `message`, optional action.
`ErrorState`	`ErrorState.tsx`	`title`, `message`, optional retry action.
`Toast`	`Toast.tsx`	`ToastContainer` renders the stack from `useApp().toasts`; do not render toasts manually.

Usage example:

```
import { Button } from '@/components/ui/Button';
import { Modal } from '@/components/ui/Modal';
import { PageHeader } from '@/components/ui/PageHeader';

<Button variant="primary" size="md" onClick={handleSave}>Save</Button>
<Modal open={open} onClose={() => setOpen(false)} title="Edit building"> ... </Modal>
<PageHeader title="Buildings" description="Manage campus buildings" />
```

Layout primitives (`AppShell`, `Sidebar`, `TopBar`) are in `src/components/layout/` and are not meant to be reused outside the authenticated app frame.

10. Theming system

The design system is built on CSS custom properties defined in `src/index.css`:

- **Light theme**: `:root`
- **Dark theme**: `[data-theme='dark']`

Theme is selected in Settings and applied by `AppContext` which sets `data-theme` on `<html>` (with a `matchMedia` listener for the `system` option).

10.1 Key tokens

TOKEN GROUP	EXAMPLES
Brand / accent	`--accent`, `--accent-hover`, `--accent-light`, `--accent-lighter`
Surfaces	`--bg`, `--surface`, `--surface-2`, `--glass-bg`, `--glass-border`
Text	`--text`, `--text-2`, `--text-3`
Borders	`--border`, `--border-2`
Status colors	`--green`, `--green-bg`, `--amber`, `--amber-bg`, `--red`, `--red-bg`
Radii	`--radius-sm`, `--radius`, `--radius-md`, `--radius-lg`, `--radius-xl`, `--radius-pill`
Shadows	`--shadow-sm`, `--shadow-md`, `--shadow-lg`, `--shadow-xl`
Layout	`--sidebar-w`, `--sidebar-collapsed-w`, `--topbar-h`
Font	`--font` (Inter stack)

> **Always theme via CSS variables** — do not hardcode colors, or dark mode will break. Use the status color pairs (`--green` / `--green-bg`, etc.) for semantic accents.

Responsive breakpoints (in `src/index.css`): `1024px`, `768px`, `375px`. The sidebar collapses into an overlay below 768px.

11. Path alias

`@/` maps to `src/` (configured in both `vite.config.ts` and `tsconfig.app.json`). Always prefer it over deep relative imports:

```
import { useApp } from '@/context/AppContext';
import { Button } from '@/components/ui/Button';
import type { Building } from '@/types';
```

12. Scripts

SCRIPT	PURPOSE
`npm run dev`	Start the Vite dev server with HMR.
`npm run build`	Type-check + production build to `dist/`.
`npm run preview`	Preview the production build locally.
`npm run typecheck`	Run `tsc --noEmit` for type errors only.
`npm run lint`	Run ESLint.

13. Integration guide

13.1 Adding a new entity type

1. **Type** — add the interface and any status unions to `src/types/index.ts`.
2. **Storage** — add a key to `STORAGE_KEYS` in `src/services/storage.ts`.
3. **Service** — add `get` / `save` / `create` / `update` / `delete` methods to `dataService` in `src/services/dataService.ts`; add seed data to `src/services/seedData.ts` and load it in `ensureSeed()`.
4. **Context** — add state + CRUD callbacks to `AppContext.tsx`, each calling `dataService`, writing an `AuditLog` via `logAction(...)`, calling `refresh()`, and showing a toast. Expose them on `AppContextValue`.
5. **UI** — create a page in `src/pages/`, add a route in `App.tsx`, and add a sidebar/topbar entry.

13.2 Calling an action from a page

```
import { useApp } from '@/context/AppContext';

export function MyBuildingForm() {
  const { createBuilding } = useApp();
  return (
    <Button onClick={() => createBuilding({
      name: 'New Block', code: 'NB', location: 'North',
      description: '...', status: 'active'
    })}>
      Add building
    </Button>
  );
}
```

The action persists to `localStorage`, writes an audit log, refreshes state, and shows a toast automatically.

13.3 Using icons

```
import { ShieldCheck } from 'lucide-react';
<ShieldCheck size={18} />
```

`lucide-react` is excluded from Vite's dep pre-bundling (`optimizeDeps.exclude` in `vite.config.ts`) — keep it that way.

14. Connecting a real backend

> The app currently uses `localStorage` and a single hard-coded demo user. To go multi-user and persistent, follow the steps below. A Supabase project is already provisioned with credentials in the environment.

1. **Supabase** is already installed (`@supabase/supabase-js`) and a project is provisioned with credentials in the environment.
2. Replace the body of `src/services/dataService.ts` with Supabase queries (keep the same method signatures so `AppContext` and pages do not change).
3. Replace the client-side auth in `dataService.authenticate` with Supabase email/password auth; keep `currentUser` in `AppContext` driven by `supabase.auth.getUser()` / `onAuthStateChange` .
4. Create tables for `buildings` , `labs` , `computers` , `alerts` , `audit_logs` , and `settings` with Row Level Security policies scoped to `auth.uid()` .
5. Move audit-log writes server-side (a Supabase trigger or an Edge Function) so they cannot be forged from the browser.

When you make this switch, the `useApp()` contract and the UI components do not need to change — only the service layer does.

15. Conventions

- **No utility-class styling inside components.** Components use semantic CSS classes from `src/index.css` ; pages compose those classes. Tailwind is configured but is primarily available for ad-hoc layout tweaks in pages.
- **One context only.** All app state flows through `AppContext` . Do not introduce a second global store.
- **Always import what you use.** Every icon, component, type, and hook must have an explicit import in the file that uses it.
- **Theme via CSS variables.** Never hardcode hex values in components.
- **Audit every mutation.** Any new create/update/delete action in `AppContext` should call `logAction(...)` so the Audit Logs page stays accurate.
- **Keep files cohesive.** A page owns its feature; shared UI belongs in `src/components/ui/` .